ReliableML: Methods for Reliable Machine-Learning Workflows

Jabulani Monchwe, Olorato Motitswane, and Mfundo Monchwe
MonchweJ.25@gmail.com

Version 0.1.1 18 October 2026

Abstract

ReliableML is a Python library for attaching data-distribution monitoring, labeled evaluation, fairness summaries, model-deterioration tracking, meta-learning, model-agnostic explanations, and statistical reason auditing to machine-learning workflows. This document summarizes the implemented methods, their interfaces, and their limits. Its central methodological qualification is that reliability is not a single scalar: distribution similarity, predictive performance, calibration, subgroup disparities, temporal trends, and explanatory stability answer different questions and must be interpreted separately.

1 Scope and workflow

Reliable model development begins with appropriate data preparation. Users may prepare data with ATTP or another tool; ReliableML accepts the resulting features and a user-selected estimator. The library does not claim that preprocessing, a high validation score, or an explanation by itself guarantees reliable deployment.

The operating workflow is:

1. Fit a model and capture a reference profile of training features.
2. Predict on incoming features and compare their distributions with the reference, optionally saving plots and machine-readable reports.
3. When labels become available, evaluate predictive quality, calibration, statistical associations, and fairness on representative labeled data.
4. Across dated labeled batches, estimate an accuracy trend.
5. Where appropriate, use out-of-fold meta-learning, local/global explanations, and a holdout reason audit to investigate stability and reliance.

2 Implemented method families

Distribution shift. Quantile-binned numerical drift statistics, categorical homogeneity tests, reference plots, and a labeled feature–target divergence diagnostic.

Evaluation. Classification metrics including multiclass Matthews correlation, confusion counts, feature–target and train–test ANOVA, MANOVA and chi-square tests, and an empirical repeated-CV Brier bias–variance decomposition.

Fairness. Statistical-parity selection-rate, equalized-odds true-/false-positive-rate, and predictive-parity positive-predictive-value gaps, with group denominators and bootstrap intervals.

Deterioration. A linear trend in accuracy across at least three dated, labeled batches and a bootstrap interval for the slope.

Meta-learning. Cross-fitted base-model meta-features and selectable, calibrated, statistical, and neural meta-classifiers. Halving grid and randomized search helpers are provided.
Explainability. A Monte Carlo permutation estimator of interventional Shapley contributions and a LIME-style local weighted linear surrogate.
Reason audit. Holdout validity, calibration, perturbation-based feature reliance, stability slices, counterfactual sensitivity, logistic association, and explanation-alignment diagnostics.
Pipeline and persistence. A scikit-learn-style wrapper stores an estimator and its fitted monitoring profile, reports prediction-time drift, and supports model persistence and exports.

3 Interpreting evidence

Each method has different data requirements. Drift monitoring can be unlabeled, but it does not establish that predictions are wrong. Performance, calibration, fairness, and deterioration assessment require labels and appropriate sampling. Counterfactual sensitivity is only meaningful when replacement values are domain-valid. Explanation agreement measures consistency, not truth or causality. Logistic odds ratios describe conditional associations, not causal effects.

Bootstrap intervals quantify sampling variation under their resampling assumptions. Current intervals generally treat rows as independent; clustered or time-dependent data may need cluster or block methods. Reported statistics should be considered alongside data provenance, leakage checks, temporal and group-aware validation, and domain review. See the standalone method documents for operational details and assumptions.

4 Minimal usage

```
from ml_reliability import ReliablePipeline

model = ReliablePipeline(estimator=classifier).fit(X_train, y_train)
predictions, report = model.predict_with_report(X_test)
evaluation = model.evaluate(X_test, y_test)
model.save("reliable-model.pkl")
```

The same fitted pipeline can be reloaded and used with input data in the original feature schema. Pickle files must only be loaded from trusted sources.

5 References

References

[1] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), 2014. https://doi.org/10.1145/2523813.

[2] D. H. Wolpert. Stacked generalization. *Neural Networks*, 5(2), 1992. https://doi.org/10.1016/S0893-6080(05)80023-1.

[3] M. Hardt, E. Price, and N. Srebro. Equality of opportunity in supervised learning. *Advances in Neural Information Processing Systems*, 2016. https://arxiv.org/abs/1610.02413.

[4] M. T. Ribeiro, S. Singh, and C. Guestrin. "Why should I trust you?" Explaining the predictions of any classifier. *Proceedings of KDD*, 2016. https://doi.org/10.1145/2939672.2939778.

[5] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 2017. https://arxiv.org/abs/1705.07874.